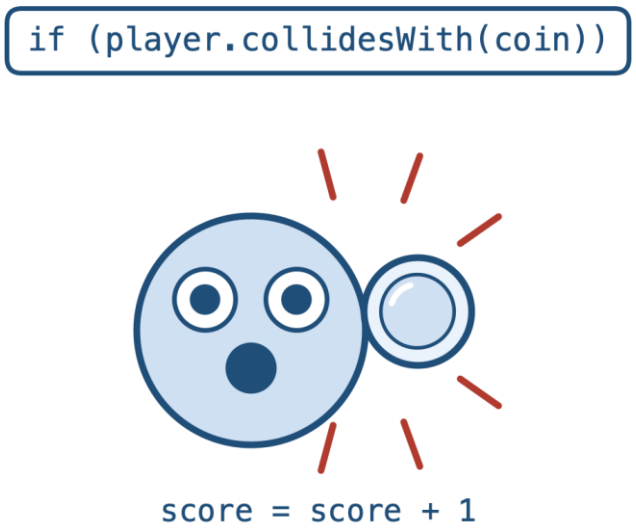
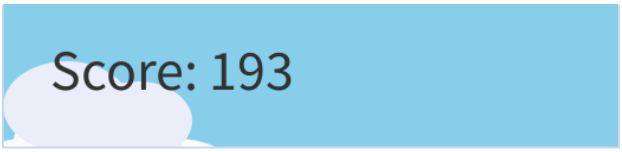
Chapter 6 - Collisions: If This, Then That

Things finally touch each other, and your program decides what happens next. This is where it becomes a game.



One coin. A few seconds. A very good score.




```
if (player.collidesWith(coin)) {
  score = score + 1;   // add a point
}
```

Chapter 5 ended with a coin drifting through your world, and touching it does nothing. What *should* happen?

Today your sketch starts paying attention. Touch the coin, and you score a point. Touch something dangerous, and the game ends.

The new idea is `if`: making something happen only when it's true. The library can tell when two things are touching, and that gives `if` something to be true about.

This chapter takes two meetings, and does it in four moves.

Part	The idea
One	**Are they touching?** A yes-or-no question, and an `if` to act on the answer.
Two	**Keep score.** A number that remembers, on the screen, one point per touch.
Three	**A game that can end.** A hazard, a true-or-false, and two screens.
Four	**Make it yours.** Harder, easier, and yours.

Parts One and Two are the first meeting. Parts Three and Four are the second.

Part One - Are they touching?

6.1 A yes-or-no question

```
player.collidesWith(coin)
```

`collidesWith()` asks one question: *are these two touching?* The library checks whether their circles overlap, and hands back `true` or `false`.

Your character has `collidesWith()` because you wrote `class MyCharacter extends Blob` back in Chapter 2. It's one of the things a Blob can do, so your character got it for free, and you never had to write it.

Remember the `30` in your `super(...)` line? In Chapter 2 you changed it, and nothing on the screen changed. This is what it's for. That number is the radius of an invisible circle around your character, called its **hitbox**, and `collidesWith()` uses that circle and only that circle. Your drawing can be any size you like.

6.2 `if`

An `if` makes something happen only when a question's answer is yes:

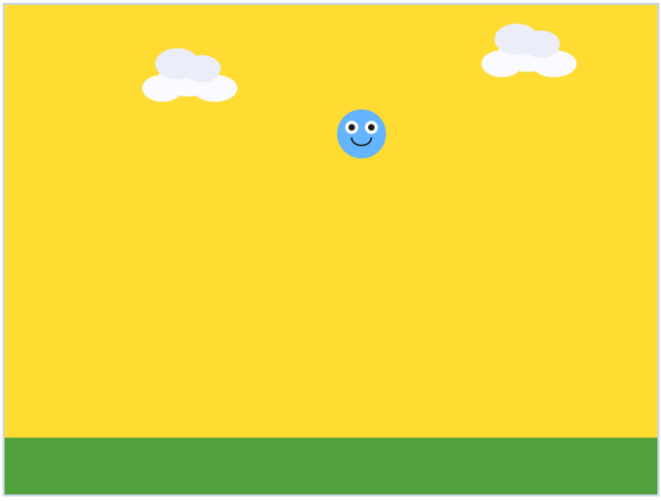
```
if (player.collidesWith(coin)) {
  background(255, 220, 50);   // only happens while they are touching
}
```

- The part in the parentheses is the **condition**, the yes-or-no question.
- The part in the braces is what happens when the condition is true.
- When it's false, Processing skips straight past the braces.

Step 1 - The gold flash

Build it. Open your sketch from Chapter 5. In `draw()`, right after your `background()` line, add the `if` above. Run it, and touch the coin.

> **Check.** The sky flashes gold while you touch the coin, and goes back to blue when you let go.



Touching the coin.

`draw()` runs sixty times a second, so it asks *are they touching?* sixty times a second. On every frame the answer is yes, the sky is gold.

Step 2 - Generous or precise

Your turn. If touching the coin feels too hard or too easy, open your `MyCharacter` tab and change the radius in the `super(...)` line. A bigger radius touches things from further away, which feels generous. A smaller one means you have to be right on top of something, which feels precise. Which do you want your game to be?

Part Two - Keep score

Step 3 - A number that remembers

Build it. Delete the gold flash. Then, at the very top of your sketch, next to your other variables, add:

```
int score = 0;      // a whole number, starts at zero
```

> **Aside: `int`**
>
> `int` is the kind of variable that holds a whole number: `0`, `7`, `-3`. No decimal point. `int score = 0;` makes a box called `score` and puts a `0` in it, on the same line.

A score has to remember something from one frame to the next. So it goes at the very **top** of your sketch, outside `setup()` and outside `draw()`. Up there, every function in your sketch can see it.

> **Watch out.** Put `int score = 0;` inside `setup()`, and the box only exists while `setup()` runs. `draw()` can't see it, and the sketch stops with:
>
> ```
> score cannot be resolved to a variable
> ```
>
> Move it back to the top.

Step 4 - The score on the screen

Build it. At the bottom of `draw()`, add three lines:

```
fill(50);                          // the text color
textSize(24);                      // how big
text("Score: " + score, 20, 36);   // what, then x, then y
```

Run it.

> **Check.** `Score: 0` in the top-left corner.



The score.

> **Aside: `text()`**
>
> `text()` draws words on the screen. You give it what to say, then where: x, then y. The `+` glues the words `"Score: "` and the number in `score` together into one piece of text. Set `fill()` **before** `text()`, or your words come out in whatever color the last shape used.

Build it. Test it by cheating. Change `int score = 0;` to `int score = 7;`, and run.

> **Check.** `Score: 7`.



The cheat.

Setting a value by hand to check that the display works is a real technique. Programmers do it all the time. Set it back to `0`.

Step 5 - A point per touch

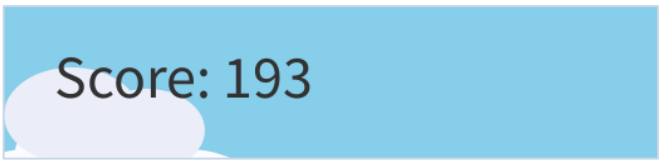
Build it. Above the three score lines, add the collision `if` again, this time with a point inside:

```
if (player.collidesWith(coin)) {
  score = score + 1;   // add a point
}
```

> **Aside: `score = score + 1`**
>
> Read it right to left. Take what's in `score`, add one, and put the answer back in `score`. The `=` means *make this equal to that*.

Run it, and sit your player on the coin. Watch the score.

> **Check.** The score races upward, as long as you're touching the coin.



A few seconds of touching.

Why is it doing that? Make a guess before you read on.

`draw()` asks *are they touching?* every frame. For as long as you sit on the coin, the answer is yes, and every yes is a point. That's a point for every frame: sixty a second, if you sit right on it.

Step 6 - One touch, one point

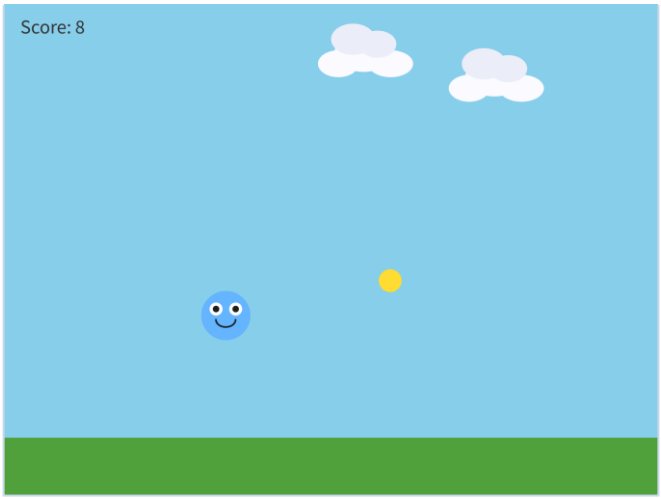
The fix is to end the touch the moment you count it. Move the coin somewhere else.

Build it. Inside the same `if`, after the point, add:

```
coin.moveTo(random(width), random(100, height - 150));  // coin jumps somewhere new
```

Run it.

> **Check.** Touch the coin, and you score one point, and the coin jumps somewhere new.



Eight coins in fifteen seconds.

You met `moveTo()` in Chapter 4, teleporting your character to a click. Here it teleports the coin, and `random()` picks the spot, so it lands somewhere new every time.

Your turn. Change the coin's speed and size, and style the score so it fits your world. The score's `fill()` and `textSize()` are yours.